

litePlay.js: a JavaScript module for *lite coding*

Concepts, tools, and research results

Felippe Barros

PhD candidate in Arts - UFES
Federal University of Espírito Santo/Brazil

May 6, 2026

Table of Contents

- 1 Introduction to litePlay.js
- 2 Study: predicting sounds

Lite coding?

- Ubiquitous music movement
- Novel music-making practice featuring musical programming with a reduced learning curve
- Tailored for non-programmers or casual participants
- Exploration of the limits of semantics for musical knowledge-sharing
- Seeks to expand participation to musicians with musical knowledge that goes beyond the acoustic-instrumental and notational logic of Western music

litePlay.js?

- **What is it?** A JavaScript module for musical *lite coding* designed by Victor Lazzarini
- **Goal:** Prioritize accessibility for beginners and casual interaction in musical programming without prescribing aesthetic ideals
- **Technology:** Powered by the Csound WASM (WebAudio) engine
- **Environments:** Online editor¹ or integration via a `<script>` tag

¹Available at g-ubimus.github.io/litePlay.js

Basic syntax

The core of the system is the `play()` function:

```
// Plays a middle C (C4) with the default piano sound  
play();  
  
// Set an instrument and play a specific note  
instrument(xylophone);  
play(E4);  
  
// Stop all sounds  
stop();
```

Notes: Uses midi values or symbolic names from C_m1 to G8 for pitch.

Parameters of an event

A musical event is defined by five optional attributes:

Structure: [what, howLoud, when, howLong, onSomething]

- **what:** The note or sound (e.g., C4, kick).
- **howLoud:** Loudness (0 to 1).
- **when:** Start time (seconds).
- **howLong:** Duration (seconds).
- **onSomething:** The instrument (e.g., violin).

Example

```
play([C4, 0.5, 0, 2, banjo])
```

Generators

Define a particular scope from which random values will be drawn:

Insert a token in place of a value

- **Pitch:** lowPitch, midPitch, highPitch (membranophones, idiophones)
- **Loudness:** soft, midLevel, loud
- **When:** now, soon, later
- **How long:** shortDur, midDur, longDur
- **On something:** instrument groups (onSynth, onVoice, onBass, etc)

Example

```
play([midPitch, soft, now, shortDur, onPlucked])
```

Sequencing events

Works with an automatic grid based on BPM.

```
setBpm(120);  
let backBeat = [kick, snare, kick, snare];  
sequencer.add(drums2, backBeat, 0.8, 1); // 1 = Quarter note  
sequencer.play();
```

Subdivisions: sub(snare, snare) divides a beat.

```
let backBeat = [  
  kick, snare, kick, sub(snare, sub(lowTom, tom, hiTom), snare)];  
  // 0 = Silence
```


How does it compare to live coding languages?

Strudel.cc: uses a highly condensed language, ideal for fast typing.

```
$: s("[bd <hh oh>]*2").bank("tr909").dec(.4)
```

litePlay.js: uses JavaScript structures, prioritizing readability.

```
function strudel(){  
  let pattern = [bassDrum1, closedHiHat, bassDrum1, openHiHat];  
  sequencer.add(drums4, pattern, loud, 0.5);  
  sequencer.play();  
}  
  
strudel();
```

Code Comprehension Study

Hypothesis: litePlay.js's simplified syntax is more readable for non-programmers than traditional Csound code.

Methodology

- **Participants:** undergraduate music students with minimal experience on programming (n=16, aged from 18 to 44 years old, avg.= 25.8)
- **Task:** to read code snippets in both programming environments and predict their sonic output.
- **Metrics:** SEQ (Single Ease Question), Accuracy scores, and SUS (System Usability Scale).

Reading code snippets

Arguably the simplest Csound instrument:

```
instr Senoide
  iDur = p3
  iAmp = 0.1
  iFreq = 440
  aSine = poscil(iAmp, iFreq)
  outall(aSine)
endin
schedule("Senoide", 0, 5)
```

Arpeggios using litePlay.js

```
lp.instrument(lp.vibraphone);
let arpejo = lp.eventList.create(
  [C4, 0.5, 0, 1],
  [E4, 0.7, 1, 1],
  [G4, 0.9, 2, 1]);
let loopArpejo = arpejo.repeat(3, 0);
lp.play(
  [G4, 1, loopArpejo, 2],
  [B4, 1, loopArpejo, 2],
  [D4, 1, loopArpejo, 2]);
```

Results: Readability and Accuracy

Data indicated a significant advantage for litePlay.js in semantic readability.

Metric	Csound	litePlay.js	p-value
SEQ (Perceived Ease)	0.32	0.51	0.036
Accuracy Score	0.26	0.46	0.027
SUS (Usability)	30	38.3	0.115

Threats to validity

Threats to external validity

- **Population validity:** small, students from the same course, non-English speakers
- **Ecological validity:** generators could give even better readability

Threats to internal validity

- **Construct validity:** other music programming languages could provide different results
- **Design confound:** IDE design, word choice, indentation, colouring can impact readability and user usability
- **Confounding variable:** a sinewave and an arpeggio are substantially different

Future perspectives

- **Dedicated IDE:**

- ✓ Read-Eval-Print Loop (REPL)
- ✓ Autocomplete
- ✓ Audio recording
- ✗ Audio upload

- **Machine listening:**

- ✓ Pitch/percussion, amplitude, duration ✗ Timbre, texture...
- ✗ Agent: suggestions, adaptations (helping with cognitive load)

Thank you!

`g-ubimus.github.io/litePlay.js/`
`g-ubimus.github.io/litePlay.docs/`